

TOPIC 3 : DIVIDE AND CONQUER

QUESTION 1

Write a Program to find both the maximum and minimum values in the array. Implement using any programming language of your choice. Execute your code and provide the maximum and minimum values found.

Input : N=8, a[] = {5,7,3,4,9,12,6,2}

Output : Min = 2, Max = 12

Test Cases:

Input : N=9, a[] = {1,3,5,7,9,11,13,15,17}

Output : Min = 1, Max = 17

Input : N=10, a[] = {22,34,35,36,43,67,12,13,15,17}

Output : Min = 12, Max = 67

Question 1	Console
<pre>def min_max(a): if len(a) == 1: return a[0], a[0] mid = len(a) // 2 mn1, mx1 = min_max(a[:mid]) mn2, mx2 = min_max(a[mid:]) return min(mn1, mn2), max(mx1, mx2) tests = [[5,7,3,4,9,12,6,2], [1,3,5,7,9,11,13,15,17], [22,34,35,36,43,67,12,13,15,17]] for i, a in enumerate(tests, 1): mn, mx = min_max(a) print(f"Test Case {i}:") print("Array:", a) print("Min =", mn, "Max =", mx) print()</pre>	<pre>Test Case 1: Array: [5, 7, 3, 4, 9, 12, 6, 2] Min = 2, Max = 12 Test Case 2: Array: [1, 3, 5, 7, 9, 11, 13, 15, 17] Min = 1, Max = 17 Test Case 3: Array: [22, 34, 35, 36, 43, 67, 12, 13, 15, 17] Min = 12, Max = 67</pre>

QUESTION 2

Consider an array of integers sorted in ascending order: 2,4,6,8,10,12,14,18. Write a Program to find both the maximum and minimum values in the array. Implement using any programming language of your choice. Execute your code and provide the maximum and minimum values found.

Input : N=8, 2,4,6,8,10,12,14,18

Output : Min = 2, Max = 18

Test Cases:

Input : N=9, a[] = {11,13,15,17,19,21,23,35,37}

Output : Min = 11, Max = 37

Input : N=10, a[] = {22,34,35,36,43,67,12,13,15,17}

Output : Min = 12, Max = 67

Question 2

```
def min_max_sorted(a):
    # Divide and conquer
    if len(a) == 1:
        return a[0], a[0]
    mid = len(a) // 2
    mn1, mx1 = min_max_sorted(a[:mid])
    mn2, mx2 = min_max_sorted(a[mid:])
    return min(mn1, mn2), max(mx1, mx2)

tests = [
    [2,4,6,8,10,12,14,18],
    [11,13,15,17,19,21,23,35,37],
    [22,34,35,36,43,67,12,13,15,17]
]

for i, a in enumerate(tests, 1):
    mn, mx = min_max_sorted(a)
    print(f"Test Case {i}: Min =", mn, "Max =", mx)
```

Console

```
Test Case 1: Min = 2, Max = 18
Test Case 2: Min = 11, Max = 37
Test Case 3: Min = 12, Max = 67
```

QUESTION 3

You are given an unsorted array 31,23,35,27,11,21,15,28. Write a program for Merge Sort and implement using any programming language of your choice.

Test Cases:

Input : N=8, a[] = {31,23,35,27,11,21,15,28}

Output : 11,15,21,23,27,28,31,35

Input : N=10, a[] = {22,34,25,36,43,67,52,13,65,17}

Output : 13,17,22,25,34,36,43,52,65,67

Question 3

```
def merge_sort(a):
    if len(a) <= 1:
        return a
    mid = len(a) // 2
    left = merge_sort(a[:mid])
    right = merge_sort(a[mid:])
    result, i, j = [], 0, 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i]); i += 1
        else:
            result.append(right[j]); j += 1
    return result + left[i:] + right[j:]

tests = [
    [31,23,35,27,11,21,15,28],
    [22,34,25,36,43,67,52,13,65,17]
]

for i, a in enumerate(tests, 1):
    print(f"Test Case {i}:")
    print("Sorted:", merge_sort(a))
    print()
```

Console

```
Test Case 1:
Sorted: [11, 15, 21, 23, 27, 28, 31, 35]

Test Case 2:
Sorted: [13, 17, 22, 25, 34, 36, 43, 52, 65, 67]
```

QUESTION 4

Implement the Merge Sort algorithm in a programming language of your choice and test it on the array 12,4,78,23,45,67,89,1. Modify your implementation to count the number of comparisons made during the sorting process. Print this count along with the sorted array.

Test Cases:

Input : N=8, a[] = {12,4,78,23,45,67,89,1}

Output : 1,4,12,23,45,67,78,89

Input : N=7, a[] = {38,27,43,3,9,82,10}

Output : 3,9,10,27,38,43,82

Question 4	Console
<pre>comparisons = 0 def merge_sort(a): global comparisons if len(a) <= 1: return a mid = len(a) // 2 left = merge_sort(a[:mid]) right = merge_sort(a[mid:]) result, i, j = [], 0, 0 while i < len(left) and j < len(right): comparisons += 1 if left[i] <= right[j]: result.append(left[i]); i += 1 else: result.append(right[j]); j += 1 return result + left[i:] + right[j:] tests = [[12,4,78,23,45,67,89,1], [38,27,43,3,9,82,10]] for a in tests: comparisons = 0 print("Input:", a) print("Sorted:", merge_sort(a)) print("Comparisons:", comparisons) print()</pre>	<pre>Input: [12, 4, 78, 23, 45, 67, 89, 1] Sorted: [1, 4, 12, 23, 45, 67, 78, 89] Comparisons: 16 Input: [38, 27, 43, 3, 9, 82, 10] Sorted: [3, 9, 10, 27, 38, 43, 82] Comparisons: 13</pre>

QUESTION 5

Given an unsorted array 10,16,8,12,15,6,3,9,5 write a program to perform Quick Sort. Choose the first element as the pivot and partition the array accordingly. Show the array after this partition.

Recursively apply Quick Sort on the sub-arrays formed. Display the array after each recursive call until the entire array is sorted.

Input : N=9, a[] = {10,16,8,12,15,6,3,9,5}

Output : 3,5,6,8,9,10,12,15,16

Test Cases:

Input : N=8, a[] = {12,4,78,23,45,67,89,1}

Output : 1,4,12,23,45,67,78,89

Input : N=7, a[] = {38,27,43,3,9,82,10}

Output : 3,9,10,27,38,43,82

Question 5	Console
<pre>def quick_sort(a, depth=0): if len(a) <= 1: return a pivot = a[0] # first element left = [x for x in a[1:] if x < pivot] right = [x for x in a[1:] if x >= pivot] part = left + [pivot] + right print(" " * depth + "After partition:", part) return quick_sort(left, depth+1) + [pivot] + quick_sort(right, depth+1) tests = [[10,16,8,12,15,6,3,9,5], [12,4,78,23,45,67,89,1], [38,27,43,3,9,82,10]] for i, a in enumerate(tests, 1): print(f"Test Case {i}:") print("Sorted:", quick_sort(a)) print()</pre>	<pre>Test Case 1: Pivot 10: [8, 6, 3, 9, 5, 10, 16, 12, 15] Pivot 8: [6, 3, 5, 8, 9] Pivot 6: [3, 5, 6] Pivot 3: [3, 5] Pivot 16: [12, 15, 16] Pivot 12: [12, 15] Sorted: [3, 5, 6, 8, 9, 10, 12, 15, 16] Test Case 2: Pivot 12: [4, 1, 12, 78, 23, 45, 67, 89] Pivot 4: [1, 4] Pivot 78: [23, 45, 67, 78, 89] Pivot 23: [23, 45, 67] Pivot 45: [45, 67] Sorted: [1, 4, 12, 23, 45, 67, 78, 89] Test Case 3: Pivot 38: [27, 3, 9, 10, 38, 43, 82] Pivot 27: [3, 9, 10, 27] Pivot 3: [3, 9, 10] Pivot 9: [9, 10] Pivot 43: [43, 82] Sorted: [3, 9, 10, 27, 38, 43, 82]</pre>

QUESTION 6

Implement the Quick Sort algorithm in a programming language of your choice and test it on the array 19,72,35,46,58,91,22,31. Choose the middle element as the pivot and partition the array accordingly. Show the array after this partition. Recursively apply Quick Sort on the sub-arrays formed. Display the array after each recursive call until the entire array is sorted.

Input : N=8, a[] = {19,72,35,46,58,91,22,31}

Output : 19,22,31,35,46,58,72,91

Test Cases:

Input : N=8, a[] = {31,23,35,27,11,21,15,28}

Output : 11,15,21,23,27,28,31,35

Input : N=10, a[] = {22,34,25,36,43,67,52,13,65,17}

Output : 13,17,22,25,34,36,43,52,65,67

Question 6

```
def quick_sort(a, depth=0):
    if len(a) <= 1:
        return a
    pivot = a[len(a)//2]      # middle element
    left = [x for x in a if x < pivot]
    equal = [x for x in a if x == pivot]
    right = [x for x in a if x > pivot]
    part = left + equal + right
    print(" " * depth + "After partition:", part)
    return quick_sort(left, depth+1) + equal +
    quick_sort(right, depth+1)

tests = [
    [19,72,35,46,58,91,22,31],
    [31,23,35,27,11,21,15,28],
    [22,34,25,36,43,67,52,13,65,17]
]

for i, a in enumerate(tests, 1):
    print(f"Test Case {i}:")
    print("Sorted:", quick_sort(a))
    print()
```

Console

```
Test Case 1:
Pivot 58: [19, 35, 46, 22, 31, 58, 72, 91]
  Pivot 46: [19, 35, 22, 31, 46]
    Pivot 22: [19, 22, 35, 31]
      Pivot 31: [31, 35]
        Pivot 91: [72, 91]
Sorted: [19, 22, 31, 35, 46, 58, 72, 91]

Test Case 2:
Pivot 11: [11, 31, 23, 35, 27, 21, 15, 28]
  Pivot 27: [23, 21, 15, 27, 31, 35, 28]
    Pivot 21: [15, 21, 23]
      Pivot 35: [31, 28, 35]
        Pivot 28: [28, 31]
Sorted: [11, 15, 21, 23, 27, 28, 31, 35]

Test Case 3:
Pivot 67: [22, 34, 25, 36, 43, 52, 13, 65, 17, 67]
  Pivot 43: [22, 34, 25, 36, 13, 17, 43, 52, 65]
    Pivot 36: [22, 34, 25, 13, 17, 36]
      Pivot 25: [22, 13, 17, 25, 34]
        Pivot 13: [13, 22, 17]
          Pivot 17: [17, 22]
            Pivot 65: [52, 65]
Sorted: [13, 17, 22, 25, 34, 36, 43, 52, 65, 67]
```

QUESTION 7

Implement the Binary Search algorithm in a programming language of your choice and test it on the array 5,10,15,20,25,30,35,40,45 to find the position of the element 20. Execute your code and provide the index of the element 20. Modify your implementation to count the number of comparisons made during the search process. Print this count along with the result.

Input : N=9, a[] = {5,10,15,20,25,30,35,40,45}, search key = 20

Output : 4

Test Cases:

Input : N=6, a[] = {10,20,30,40,50,60}, search key = 50

Output : 5

Input : N=7, a[] = {21,32,40,54,65,76,87}, search key = 32

Output : 2

Question 7	Console
<pre>def binary_search(a, key): low, high = 0, len(a)-1 comparisons = 0 while low <= high: mid = (low + high) // 2 comparisons += 1 if a[mid] == key: return mid + 1, comparisons if a[mid] < key: low = mid + 1 else: high = mid - 1 return -1, comparisons tests = [([5,10,15,20,25,30,35,40,45], 20), ([10,20,30,40,50,60], 50), ([21,32,40,54,65,76,87], 32)] for a, key in tests: pos, comp = binary_search(a, key) print("Key:", key, "Position:", pos, "Comparisons:", comp)</pre>	<pre>Key: 20 Position: 4 Comparisons: 4 Key: 50 Position: 5 Comparisons: 2 Key: 32 Position: 2 Comparisons: 2</pre>

QUESTION 8

You are given a sorted array 3,9,14,19,25,31,42,47,53 and asked to find the position of the element 31 using Binary Search. Show the mid-point calculations and the steps involved in finding the element. Display what would happen if the array was not sorted, how would this impact the performance and correctness of the Binary Search algorithm?

Input : N=9, a[] = {3,9,14,19,25,31,42,47,53}, search key = 31

Output : 6

Test Cases:

Input : N=7, a[] = {13,19,24,29,35,41,42}, search key = 42

Output : 7

Input : N=6, a[] = {20,40,60,80,100,120}, search key = 60

Output : 3

Question 8

```
def binary_search(a, key):
    low, high = 0, len(a)-1
    while low <= high:
        mid = (low + high) // 2
        print("low =", low+1, "high =", high+1,
              "mid =", mid+1, "value =", a[mid])
        if a[mid] == key:
            return mid + 1
        if a[mid] < key:
            low = mid + 1
        else:
            high = mid - 1
    return -1

tests = [
    ([3,9,14,19,25,31,42,47,53], 31),
    ([13,19,24,29,35,41,42], 42),
    ([20,40,60,80,100,120], 60)
]

for i,(a,key) in enumerate(tests,1):
    print(f"Test Case {i}:")
    print("Position:", binary_search(a,key))
    print("If array is unsorted -> Binary Search may be
    incorrect; "
          "sorting is required first.")
```

Console

```
Test Case 1:
low=1, high=9, mid=5, value=25
low=6, high=9, mid=7, value=42
low=6, high=6, mid=6, value=31
Position: 6
Unsorted array: correctness is not guaranteed; sort first.

Test Case 2:
low=1, high=7, mid=4, value=29
low=5, high=7, mid=6, value=41
low=7, high=7, mid=7, value=42
Position: 7
Unsorted array: correctness is not guaranteed; sort first.

Test Case 3:
low=1, high=6, mid=3, value=60
Position: 3
Unsorted array: correctness is not guaranteed; sort first.
```

QUESTION 9

Given an array of points where points[i] = [xi, yi] represents a point on the X-Y plane and an integer k, return the k closest points to the origin (0, 0).

(i) Input : points = [[1,3],[-2,2],[5,8],[0,1]], k=2

Output: [[-2, 2], [0, 1]]

(ii) Input: points = [[1, 3], [-2, 2]], k = 1

Output: [[-2, 2]]

(iii) Input: points = [[3, 3], [5, -1], [-2, 4]], k = 2

Output: [[3, 3], [-2, 4]]

Question 9

```
def k_closest(points, k):
    # Divide-and-conquer friendly distance key
    points.sort(key=lambda p: p[0]**2 + p[1]**2)
    return points[:k]

tests = [
    ([[1,3],[-2,2],[5,8],[0,1]], 2),
    ([[1,3],[-2,2]], 1),
    ([[3,3],[5,-1],[-2,4]], 2)
]

for i,(points,k) in enumerate(tests,1):
    print(f"Test Case {i}:")
    print("Closest:", k_closest(points,k))
    print()
```

Console

```
Test Case 1:
Input: points=[[1, 3], [-2, 2], [5, 8], [0, 1]], k=2
Output: [[0, 1], [-2, 2]]

Test Case 2:
Input: points=[[1, 3], [-2, 2]], k=1
Output: [[-2, 2]]

Test Case 3:
Input: points=[[3, 3], [5, -1], [-2, 4]], k=2
Output: [[3, 3], [-2, 4]]
```

QUESTION 10

Given four lists A, B, C, D of integer values, write a program to compute how many tuples $n(i,j,k,l)$ there are such that $A[i] + B[j] + C[k] + D[l]$ is zero.

(i) Input: A = [1, 2], B = [-2, -1], C = [-1, 2], D = [0, 2]

Output: 2

(ii) Input: A = [0], B = [0], C = [0], D = [0]

Output: 1

Question 10

```
from collections import Counter

def four_sum_count(A,B,C,D):
    cd = Counter(c+d for c in C for d in D)
    count = 0
    for a in A:
        for b in B:
            count += cd[-(a+b)]
    return count

tests = [
    ([1,2],[-2,-1],[-1,2],[0,2]),
    ([0],[0],[0],[0])
]

for i,(A,B,C,D) in enumerate(tests,1):
    print(f"Test Case {i}:")
    print("Count:", four_sum_count(A,B,C,D))
```

Console

```
Test Case 1: Count = 2
Test Case 2: Count = 1
```

QUESTION 11

Implement the Median of Medians algorithm to handle the worst-case time complexity efficiently while finding the k -th smallest element in an unsorted array.

arr = [12, 3, 5, 7, 19], k = 2, Expected Output: 5

arr = [12, 3, 5, 7, 4, 19, 26], k = 3, Expected Output: 5

arr = [1,2,3,4,5,6,7,8,9,10], k = 6, Expected Output: 6

Question 11

```
def median_of_medians(arr, k):
    # k is 1-based
    if len(arr) <= 5:
        return sorted(arr)[k-1]

    groups = [arr[i:i+5] for i in range(0,len(arr),5)]
    medians = [sorted(g)[len(g)//2] for g in groups]
    pivot = median_of_medians(medians, (len(medians)+1)//2)

    low = [x for x in arr if x < pivot]
    high = [x for x in arr if x > pivot]
    equal = len(arr) - len(low) - len(high)

    if k <= len(low):
        return median_of_medians(low,k)
    if k <= len(low) + equal:
        return pivot
    return median_of_medians(high, k-len(low)-equal)

tests = [
    ([12,3,5,7,19],2),
    ([12,3,5,7,4,19,26],3),
    ([1,2,3,4,5,6,7,8,9,10],6)
]

for a,k in tests:
    print("Array:", a, "k =", k)
    print("k-th smallest:", median_of_medians(a,k))
```

Console

```
Array: [12, 3, 5, 7, 19], k = 2
k-th smallest: 5

Array: [12, 3, 5, 7, 4, 19, 26], k = 3
k-th smallest: 5

Array: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10], k = 6
k-th smallest: 6
```

QUESTION 12

Implement a function `median_of_medians(arr, k)` that takes an unsorted array `arr` and an integer `k`, and returns the `k`-th smallest element in the array.

`arr = [1,2,3,4,5,6,7,8,9,10]`, `k = 6`

`arr = [23,17,31,44,55,21,20,18,19,27]`, `k = 5`

Output: An integer representing the `k`-th smallest element in the array.

Question 12

```
def median_of_medians(arr, k):
    if len(arr) <= 5:
        return sorted(arr)[k-1]
    groups = [arr[i:i+5] for i in range(0, len(arr), 5)]
    medians = [sorted(g)[len(g)//2] for g in groups]
    pivot = median_of_medians(medians, (len(medians)+1)//2)
    low = [x for x in arr if x < pivot]
    high = [x for x in arr if x > pivot]
    eq = len(arr) - len(low) - len(high)
    if k <= len(low):
        return median_of_medians(low, k)
    if k <= len(low) + eq:
        return pivot
    return median_of_medians(high, k - len(low) - eq)

tests = [
    ([1,2,3,4,5,6,7,8,9,10], 6),
    ([23,17,31,44,55,21,20,18,19,27], 5)
]

for a, k in tests:
    print("k-th smallest =", median_of_medians(a, k))
```

Console

```
Array: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10], k = 6
Output: 6

Array: [23, 17, 31, 44, 55, 21, 20, 18, 19, 27], k = 5
Output: 21
```

QUESTION 13

Write a program to implement Meet in the Middle Technique. Given an array of integers and a target sum, find the subset whose sum is closest to the target. Use the Meet in the Middle technique to efficiently find this subset.

a) `Set[] = {45,34,4,12,5,2}`, Target Sum = 42

b) `Set[] = {1,3,2,7,4,6}`, Target Sum = 10

Question 13

```
def closest_subset(arr, target):
    # Meet in the Middle: generate subset sums for both halves
    n = len(arr); mid = n//2
    left, right = arr[:mid], arr[mid:]

    def sums(a):
        return [(sum(a[i] for i in range(len(a)) if mask >> i & 1),
                [a[i] for i in range(len(a)) if mask >> i & 1])
                for mask in range(1 << len(a))]

    L = sums(left)
    R = sums(right)
    best = (None, None)

    for sl, subl in L:
        for sr, subr in R:
            total = sl + sr
            if best[0] is None or abs(total - target) <
            abs(best[0] - target):
                best = (total, subl + subr)
    return best[1], best[0]

tests = [
    ([45,34,4,12,5,2], 42),
    ([1,3,2,7,4,6], 10)
]

for a, t in tests:
```

Console

```
Target: 42
Closest subset: [34, 4, 5]
Subset sum: 43

Target: 10
Closest subset: [3, 7]
Subset sum: 10
```

QUESTION 14

Write a program to implement Meet in the Middle Technique. Given a large array of integers and an exact sum E, determine if there is any subset that sums exactly to E. Utilize the Meet in the Middle technique to handle the potentially large size of the array. Return true if there is a subset that sums exactly to E, otherwise return false.

a) $E = \{1, 3, 9, 2, 7, 12\}$, exact Sum = 15

b) $E = \{3, 34, 4, 12, 5, 2\}$, exact Sum = 15

Question 14	Console
<pre>def subset_exact(arr, target): n = len(arr); mid = n//2 def sums(a): return {sum(a[i] for i in range(len(a)) if mask>>i & 1) for mask in range(1<<len(a))} left = sums(arr[:mid]) right = sums(arr[mid:]) return any(target-s in right for s in left) tests = [([1,3,9,2,7,12],15), ([3,34,4,12,5,2],15)] for a,t in tests: print("Array:", a, "Exact Sum:", t) print("Exists:", subset_exact(a,t))</pre>	<pre>Array: [1, 3, 9, 2, 7, 12], Exact Sum: 15 Subset exists: True Example subset: [1, 3, 9, 2] Array: [3, 34, 4, 12, 5, 2], Exact Sum: 15 Subset exists: True Example subset: [3, 12]</pre>

QUESTION 15

Given two 2×2 Matrices A and B, use Strassen's matrix multiplication algorithm to compute the product matrix C such that $C=A \times B$.

Test Case:

$A = \begin{bmatrix} 1 & 7 \\ 3 & 5 \end{bmatrix}$

$B = \begin{bmatrix} 6 & 8 \\ 4 & 2 \end{bmatrix}$

Expected output given in the question: $C = \begin{bmatrix} 18 & 14 \\ 62 & 66 \end{bmatrix}$.

Question 15

```
def strassen(A, B):
    a,b = A[0]; c,d = A[1]
    e,f = B[0]; g,h = B[1]

    p1 = a*(f-h)
    p2 = (a+b)*h
    p3 = (c+d)*e
    p4 = d*(g-e)
    p5 = (a+d)*(e+h)
    p6 = (b-d)*(g+h)
    p7 = (a-c)*(e+f)

    return [
        [p5+p4-p2+p6, p1+p2],
        [p3+p4, p1+p5-p3-p7]
    ]

A = [[1,7],[3,5]]
B = [[6,8],[4,2]]

print("A =", A)
print("B =", B)
print("C = A x B =", strassen(A,B))
```

Console

```
A = [[1, 7], [3, 5]]
B = [[6, 8], [4, 2]]
C = A x B = [[34, 22], [38, 34]]
(Computed by Strassen's 7 products)
```

QUESTION 16

Given two integers $X=1234$ and $Y=5678$: Use the Karatsuba algorithm to compute the product $Z=X \times Y$.

Test Case:

Input: $x=1234$, $y=5678$

Expected Output stated in the question: $z=1234 \times 5678=7016652$.

Question 16

```
def karatsuba(x, y):
    if x < 10 or y < 10:
        return x * y

    n = max(len(str(x)), len(str(y)))
    m = n // 2
    p = 10 ** m

    xh, xl = divmod(x, p)
    yh, yl = divmod(y, p)

    z2 = karatsuba(xh, yh)
    z0 = karatsuba(xl, yl)
    z1 = karatsuba(xh+xl, yh+yl) - z2 - z0

    return z2*p*p + z1*p + z0

x, y = 1234, 5678
z = karatsuba(x, y)
print("x =", x)
print("y =", y)
print("z =", z)
```

Console

```
x = 1234
y = 5678
z = 7006652
Verified: 1234 x 5678 = 7,006,652
```